Reporte

Nombre: Jorge Luis Molina García	**Matrícula**: 2147723
Nombre: Jonathan Alexander Reyes Garcia	**Matrícula**: 2146387
Nombre: Leonardo Davila Gongora	**Matrícula**: 2145945

Nombre del curso: Componentes de Lógica de Negocios (CLN) – ITS	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad: PIA (Unidades 2 a 5)	**Actividad**: Proyecto Integrador de Aprendizaje (PIA): Hotel Costa Azul (Caso 09), API de Reservaciones
Fecha: 05/11/2026	

Bibliografía (Formato APA)

Gómez Marroquín, M. Á. (s. f.). Proyecto Integrador de Aprendizaje (PIA): Componentes de Lógica de Negocios, AD 2026 [Material de curso]. Facultad de Ingeniería Mecánica y Eléctrica, UANL.

Evans, E. (2003). Domain-Driven Design: Tackling complexity in the heart of software. Addison-Wesley.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Pearson.

Object Management Group. (2011). Business Process Model and Notation (BPMN), version 2.0. https://www.omg.org/spec/BPMN/2.0/

Ramírez, S. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com

SQLAlchemy. (s. f.). SQLAlchemy 2.0 documentation. https://docs.sqlalchemy.org

pytest-dev. (s. f.). pytest documentation. https://docs.pytest.org

Docker Inc. (s. f.). Docker Compose documentation. https://docs.docker.com/compose/

GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/actions

Render. (s. f.). Render documentation: Blueprints. https://render.com/docs

Wiggins, A. (2017). The twelve-factor app. https://12factor.net

Objetivo:

Diseñar, construir, probar y documentar el núcleo de negocio de Hotel Costa Azul (Caso 09) como un proyecto integrador: requerimientos (BRD), historias de usuario con criterios Gherkin, proceso BPMN, diseño guiado por el dominio (DDD) con los patrones Specification y State Machine, una API REST documentada con OpenAPI/Swagger, persistencia en base de datos, pruebas con cobertura de al menos 80 %, Docker Compose (base de datos, aplicación y pruebas), integración y despliegue continuos con GitHub Actions y un despliegue público. El PIA integra el trabajo de las unidades anteriores y las reglas del

laboratorio de la Actividad 5 (tope de crédito y cancelación a 48 h) en un solo proyecto; todo lo que cambió respecto de lo entregado antes está marcado como CAMBIO.

Procedimiento:

1. Resumen de la entrega y estado de verificación

El sistema cubre la reservación, el check-in, la estancia, el check-out y la limpieza de habitaciones, con las excepciones de cancelación, no-show y cambio de habitación. Resuelve los cuatro problemas del caso: Recepción ve de inmediato lo que Limpieza libera, el cupo no se vende de más, cada cambio de habitación queda en un historial y cada estancia tiene una cuenta única. Se parte del proyecto entregado anteriormente (PIA_Hotel_Costa_Azul) y se le integran las reglas de la Actividad 5.

Qué está verificado y qué falta ejecutar. El proyecto se preparó en un entorno sin FastAPI, SQLAlchemy, Ruff ni Docker. Lo que depende de esas herramientas está escrito y revisado, pero se debe ejecutar en el equipo del grupo; lo demás sí se ejecutó.

Elemento	Estado	Detalle
Dominio, servicios, permisos, datos del PIA, errores HTTP y trazabilidad	Verificado	186 pruebas unitarias pasaron (pytest). Cobertura aproximada medida con el módulo trace: 1,215 de 1,221 líneas (99.5 %) de los módulos que no dependen de FastAPI ni SQLAlchemy.
120 casos y 20 transiciones del PIA	Verificado	Se ejecutaron sobre el sistema y se compararon con un cálculo independiente sobre los CSV.
Persistencia SQL y API (FastAPI)	Por ejecutar	23 pruebas de integración escritas (8 de persistencia y 15 de la API); no se pudieron correr en el entorno de elaboración.
Ruff y cobertura oficial (pytest-cov)	Por ejecutar	Sin los módulos de API y persistencia, la cobertura total estimada baja a cerca de 70 %; con las pruebas de integración se espera superar 80 %.
Docker Compose y GitHub Actions	Por ejecutar	Archivos escritos; requieren ejecutarse en el equipo y en el repositorio.
Despliegue público y capturas	Por hacer	El render.yaml está listo; falta crear el servicio con la cuenta del equipo y tomar las capturas P1 a P8 (sección 11).

2. Cambios respecto a lo entregado anteriormente (marcados)

Todo lo que se modificó o agregó se marca con CAMBIO en los documentos y en el código (comentarios «CAMBIO PIA»). El detalle completo está en docs/CAMBIOS_RESPECTO_A_ENTREGA_ANTERIOR.md.

#	CAMBIO	Dónde	Motivo
1	Tope de crédito de $3,000 MXN en consumos (sin hospedaje, menos pagos); se supera con pago parcial o autorización de Administración. Error nuevo CreditoExcedidoError (409).	src/domain/ cuenta_estancia.py y capas superiores	Regla del BRD del equipo y de la Actividad 5 que el PIA anterior no tenía.

#	CAMBIO	Dónde	Motivo
2	Penalidad de cancelación: gratis con ≥ 48 h; si no, una noche; excepción del Jefe de Reservaciones/Administración. La respuesta incluye penalidad_cancelacion y el evento ReservacionCancelada lleva la penalidad.	src/domain/ reservacion.py y capas superiores	Misma razón.
3	POST /cancelar acepta un cuerpo opcional (fecha_llegada, autorizacion_jefe) y POST /cargos acepta autorizacion_administracion; sin cuerpo todo funciona como antes.	src/api/app.py, docs/api/openapi.yaml	Compatibilidad hacia atrás.
4	BRD (RF-05, RF-08, alcance, supuestos, riesgos y pregunta abierta 1), historias US-05 y US-08 (5 escenarios Gherkin nuevos), matriz de trazabilidad y glosario (2 términos).	docs/	Reflejar los cambios 1 y 2.
5	+13 pruebas unitarias y +2 de integración (186 unitarias en total; antes 173).	tests/	Probar los cambios 1 y 2.
6	Informe rehecho con la portada FIME y los tres integrantes; presentación rehecha en el formato FIME.	entregables/	Pedido del equipo.

No cambió: la arquitectura, los 10 ADR, los diagramas, la máquina de estados, las Specifications, la persistencia, Docker, CI/CD y los resultados de los 120 casos y 20 transiciones. **Sigue sin implementarse:** la expiración de la pre-reserva a 24 h (sí se hizo en el laboratorio de la Actividad 5, E9, donde el estado se llamaba PRE_RESERVA; en el PIA el estado inicial es SOLICITADA, como en el paquete), el no-show con cargo, la salida anticipada, la extensión y la llegada anticipada.

3. Análisis: requerimientos de negocio (BRD)

El documento completo está en docs/BRD.md. Lo que el caso no define se marcó como supuesto o pregunta abierta; no se inventaron políticas.

3.1 Contexto. Hotel Costa Azul tiene 120 habitaciones. Sus ingresos vienen del hospedaje y de servicios adicionales, con tarifas que dependen de la temporada, el tipo de habitación y las condiciones comerciales. El proceso central es reservación, check-in, estancia, check-out y limpieza.

3.2 Problemas y oportunidades

#	Problema observado	Oportunidad
P1	Recepción no ve de inmediato qué habitaciones libera Limpieza.	Un solo estatus por habitación, actualizado por Limpieza y visible para todos.
P2	Riesgo de inconsistencias entre la disponibilidad mostrada y la real.	Cupo por categoría que se descuenta al confirmar y se devuelve al cancelar.
P3	Los cambios de habitación son difíciles de rastrear.	Historial de cada cambio (origen, destino, motivo, responsable, fecha).
P4	Los cargos de la estancia están dispersos.	Una cuenta única por estancia, con el empleado que capturó cada cargo.

3.3 Stakeholders

Rol	Interés
Huésped	Reservar, hospedarse y pagar sin errores.
Reservaciones	Confirmar, autorizar y cancelar reservaciones con disponibilidad real.

Rol	Interés
Recepción	Check-in, cargos, cambios de habitación y check-out; ver el estatus de habitaciones.
Limpieza	Marcar habitaciones como listas.
Administración	Supervisar ocupación, sacar habitaciones de servicio y auditar.

3.4 Alcance. Dentro: consulta de categorías y cupo; solicitud, autorización, confirmación, cancelación (con penalidad a 48 h) y no-show; check-in con identidad y garantía validada por un servicio externo; cargos y cuenta de estancia (con tope de crédito); cambio de habitación con historial; check-out con liquidación; ciclo de limpieza; habitación fuera de servicio; reporte de ocupación; beneficio preferente; control de acceso por rol; API REST, PostgreSQL, Docker y CI/CD. Diferido: salida anticipada, extensión, llegada anticipada, no-show con cargo, expiración de la pre-reserva y tarifas por temporada. Fuera de alcance: contabilidad, nómina, facturación fiscal, pagos reales con tarjeta e interfaz gráfica.

3.5 Necesidades

ID	Necesidad
N1	Consultar categorías de habitación con su tarifa y cupo disponible.
N2	Solicitar y confirmar reservaciones sin vender más cupo del que existe.
N3	Hacer el check-in verificando identidad y garantía, y entregando una habitación lista.
N4	Registrar los consumos de la estancia en una sola cuenta.
N5	Revisar y liquidar los cargos en el check-out.
N6	Mandar la habitación a limpieza al terminar la estancia.
N7	Que Recepción vea de inmediato cuando Limpieza libera una habitación.
N8	Rastrear los cambios de habitación.
N9	Gestionar cancelaciones y no-show devolviendo el cupo.
N12	Sacar de servicio una habitación y volver a activarla.
N14	Que Administración vea la ocupación del hotel.
N15	Reconocer a los huéspedes preferentes con un beneficio (regla del paquete de pruebas del PIA, por confirmar con el negocio).
N16	Que cada rol solo pueda ejecutar las operaciones que le corresponden.
N10, N11, N13	Salida anticipada, extensión y llegada anticipada (diferidas).

3.6 Requisitos funcionales (CAMBIO en RF-05 y RF-08)

ID	Requisito	Necesidad
RF-01	El sistema lista las categorías con tarifa, cupo y si están habilitadas.	N1
RF-02	El sistema registra una solicitud de reservación con una o más líneas (categoría y cantidad) y toma la tarifa del catálogo.	N2
RF-03	Reservaciones puede autorizar una reservación solicitada.	N2
RF-04	Una reservación solo se confirma si el huésped está activo, tiene detalle, está autorizada, sus categorías están habilitadas y hay cupo; al confirmar se descuenta el cupo.	N2
RF-05	Una reservación solicitada o confirmada puede cancelarse; si estaba confirmada, el cupo se devuelve. **[CAMBIO]** Si estaba confirmada y la llegada es en menos de 48 h, se cobra 1 noche de penalidad, salvo excepción autorizada por el Jefe de Reservaciones/Administración.	N9

ID	Requisito	Necesidad
RF-06	Una reservación confirmada puede marcarse como no-show; el cupo se devuelve.	N9
RF-07	El check-in exige identidad verificada y una garantía aprobada por el servicio externo; entrega una habitación disponible de la categoría reservada y abre la cuenta con el hospedaje.	N3
RF-08	Durante la estancia se registran cargos con concepto, monto y empleado. **[CAMBIO]** Los consumos (sin hospedaje) tienen un tope de crédito de $3,000 MXN; se supera con pago parcial previo o autorización de Administración.	N4
RF-09	Durante la estancia se puede cambiar de habitación; la anterior pasa a limpieza y el cambio queda en el historial.	N8
RF-10	El check-out registra el pago y cierra la cuenta solo si no queda saldo; la habitación pasa a pendiente de limpieza.	N5, N6
RF-11	Solo Limpieza puede marcar una habitación pendiente de limpieza como disponible, y el cambio queda visible de inmediato.	N7
RF-12	Administración puede sacar de servicio una habitación (no ocupada) y reactivarla.	N12
RF-13	Cualquier rol puede consultar una reservación; el sistema indica si requiere revisión.	N2
RF-14	Los roles pueden consultar el estatus de todas las habitaciones.	N7
RF-15	Recepción y Administración pueden consultar la cuenta de una estancia.	N4
RF-16	Se puede consultar el historial de cambios de una reservación.	N8
RF-17	Administración obtiene un reporte de ocupación con habitaciones y reservaciones por estado.	N14
RF-18	Al confirmar, si el huésped es preferente, el total llega al umbral y no hay promoción activa, se otorga el beneficio preferente.	N15
RF-19	Cada operación valida el rol; Administración puede consultar los eventos del sistema.	N16

3.7 Requisitos no funcionales

ID	Requisito
RNF-01	Consistencia: cada caso de uso guarda todos sus cambios o ninguno (Unit of Work).
RNF-02	Mantenibilidad: capas separadas; el dominio no depende de frameworks ni de la base de datos.
RNF-03	Verificabilidad: cobertura mínima de 80% medida con pytest-cov.
RNF-04	Configuración por variables de entorno; sin secretos en el código ni en GitHub.
RNF-05	Despliegue reproducible con Docker y Docker Compose.
RNF-06	Documentación de la API generada automáticamente (OpenAPI en /docs).
RNF-07	Los errores devuelven códigos HTTP coherentes: 400, 403, 404, 409, 422 y 503.
RNF-08	Importes monetarios con precisión decimal (no con números de punto flotante).

3.8 Restricciones. Hotel de 120 habitaciones; una sola moneda (MXN); Python, FastAPI, SQLAlchemy y PostgreSQL; los datos del paquete del PIA son ficticios y solo sirven para probar.

3.9 Supuestos principales. El cupo de cada categoría (capacidad_o_stock) son habitaciones-noche y se descuenta al confirmar; el total de una reservación es la suma de tarifa por cantidad; el umbral del beneficio preferente (1,500 MXN) viene del paquete del PIA y es configurable; la matriz de permisos por rol es una propuesta de diseño; **(CAMBIO)** el tope de crédito se calcula sobre los consumos

sin el hospedaje menos lo pagado, y la autorización de Administración y la excepción del Jefe de Reservaciones se modelaron como indicadores en la petición.

3.10 Criterios de éxito

Criterio	Cómo se verifica
Cero confirmaciones por encima del cupo	Pruebas de confirmación sin cupo y de la última suite.
Recepción ve el estatus actualizado de inmediato	Prueba del ciclo check-out → limpieza → disponible y consulta de habitaciones.
100% de los cambios de habitación con historial	Pruebas de cambio de habitación e historial.
Una cuenta única por estancia	Pruebas de cuenta, cargos y cierre.
Ninguna operación guarda cambios a medias	Pruebas de rechazo con reversión.
Los 120 casos y las 20 transiciones del PIA coinciden con un cálculo independiente	tests/unit/test_datos_pia.py
Cobertura mínima de 80%	pytest-cov en CI.

3.11 Preguntas abiertas para el negocio. (1) Pregunta respondida por el BRD del equipo (CAMBIO): plazo de cancelación de 48 h y penalidad de una noche. Siguen abiertas: cobro por no-show, criterio para autorizar una reservación, tipos de garantía, cambio de categoría, beneficio concreto del huésped preferente, salida anticipada y extensión, y reportes adicionales.

4. Historias de usuario con criterios de aceptación (Gherkin)

Las 19 historias implementadas están en docs/historias_usuario.md, cada una con escenarios Gherkin de éxito y de rechazo (más de 30 escenarios). Las historias US-20 a US-22 (salida anticipada, extensión y llegada anticipada) quedan en el backlog.

ID	Historia	Rol	Resultado esperado
US-01	Consultar categorías y cupo	Reservaciones	Ofrecer solo lo que se puede reservar.
US-02	Solicitar una reservación	Reservaciones	Reservación SOLICITADA con tarifa del catálogo.
US-03	Autorizar una reservación	Reservaciones	Reservación autorizada.
US-04	Confirmar una reservación	Reservaciones	Confirmada y cupo descontado; rechazo con motivo.
US-05	Cancelar una reservación (CAMBIO: penalidad 48 h)	Reservaciones	Cancelada, cupo devuelto y penalidad si es tardía.
US-06	Registrar un no-show	Recepción	NO_SHOW y cupo devuelto.
US-07	Hacer el check-in	Recepción	Habitación ocupada y cuenta abierta.
US-08	Registrar cargos (CAMBIO: tope de crédito)	Recepción	Cargo en la cuenta; rechazo si supera el tope.
US-09	Cambiar de habitación	Recepción	Historial y la anterior a limpieza.
US-10	Hacer el check-out	Recepción	Cuenta cerrada y habitación a limpieza.
US-11	Marcar habitación disponible	Limpieza	Estatus DISPONIBLE y evento publicado.
US-12	Fuera de servicio y reactivar	Administración	Habitación no ofrecida mientras no sirva.

ID	Historia	Rol	Resultado esperado
US-13	Consultar una reservación	Reservaciones	Detalle y requiere_revision.
US-14	Ver estatus de habitaciones	Recepción	Estatus actualizado de inmediato.
US-15	Consultar la cuenta	Recepción	Total, pagado y saldo.
US-16	Historial de cambios	Administración	Auditoría de cambios.
US-17	Reporte de ocupación	Administración	Conteos y porcentaje de ocupación.
US-18	Beneficio preferente	Administración	Beneficio al confirmar si cumple.
US-19	Acceso por rol y eventos	Administración	403 sin permiso; auditoría de eventos.

Ejemplo de criterios de aceptación (US-05, escenarios CAMBIO):

```
Scenario: Cancelación tardía
  Given una reservación CONFIRMADA con llegada en menos de 48 horas
  When Reservaciones la cancela sin autorización especial
  Then se cobra una noche (la tarifa de la primera línea) como penalidad

Scenario: Excepción del Jefe de Reservaciones o Administración
  Given una cancelación tardía con autorización del Jefe o de Administración
  When se cancela
  Then la penalidad es cero
```

Ejemplo (US-08, escenario CAMBIO):

```
Scenario: Consumo que supera el tope de crédito
  Given consumos acumulados de 3,000 MXN y ningún pago parcial
  When Recepción intenta registrar otro cargo
  Then el sistema responde 409 y el cargo no se guarda
```

5. Proceso de negocio (BPMN)

El diagrama muestra cinco carriles (Huésped, Reservaciones, Recepción, Limpieza y Administración), las decisiones del proceso, las excepciones (rechazo, cancelación, no-show y check-in rechazado) y los objetos de datos (Reservación, Cuenta de estancia y Estatus de habitación).

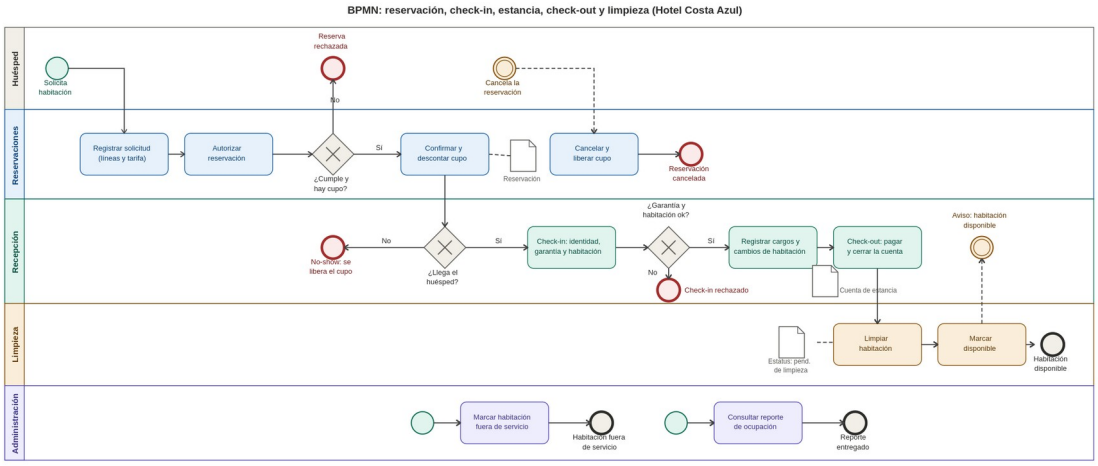


Figura 1. BPMN del proceso principal (docs/architecture/diagramas/).

6. Diseño del dominio (DDD)

6.1 Mapa de dominio. Hay cuatro agregados con su raíz: Reservacion (con CambioDeHabitacion), Habitacion, CategoriaHabitacion y CuentaEstancia (con Cargo). Los demás conceptos son value objects, specifications y eventos de dominio.

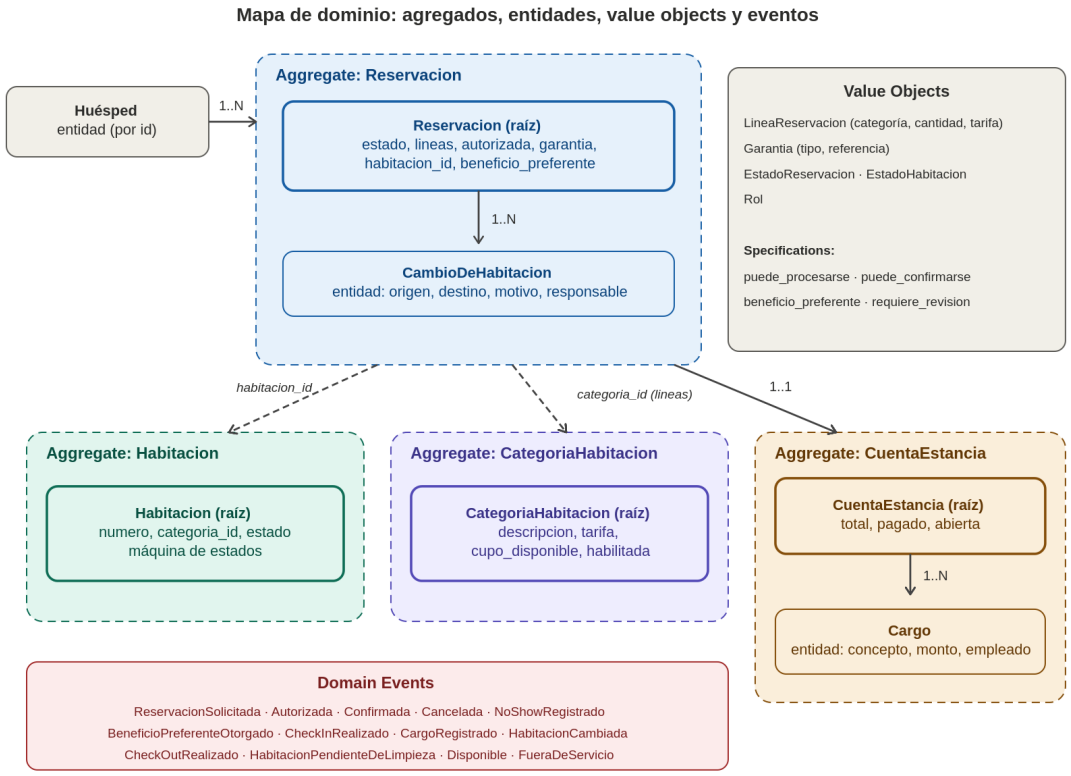


Figura 2. Mapa de dominio: agregados, value objects, specifications y eventos.

6.2 Reglas que protege el dominio

ID	Regla	Dónde
R1	Una habitación solo se entrega si está disponible.	Habitacion.entregar
R2	No se confirma más cupo del que existe.	Reservacion.confirmar
R3	Cada operación depende del estado de la reservación.	Máquina de estados
R4	El check-in exige identidad y garantía aprobada.	Reservacion.registrar_check_in
R5	Solo se agregan cargos a una cuenta abierta y mayores a cero.	CuentaEstancia.registrar_cargo
R6	No se cierra la cuenta con saldo pendiente.	CuentaEstancia.cerrar
R7 (CAMBIO)	Los consumos no superan $3,000 MXN, salvo pago parcial o autorización de Administración.	CuentaEstancia.registrar_cargo
R8	Todo cambio de habitación deja registro.	Reservacion.cambiar_habitacion
R9	La habitación liberada queda pendiente de limpieza; solo Limpieza la libera.	Máquina de estados y permisos
R10	Una habitación fuera de servicio no acepta reservas ni entregas.	Habitacion
R11 (CAMBIO)	Cancelar una reservación confirmada con menos de 48 h cobra una noche, salvo excepción autorizada.	Reservacion.cancelar

6.3 Máquinas de estados. La reservación sigue SOLICITADA → CONFIRMADA → CHECK_IN → CHECK_OUT, con CANCELADA y NO_SHOW como salidas alternativas; la habitación sigue DISPONIBLE → OCUPADA → PENDIENTE_DE_LIMPIEZA → DISPONIBLE, con FUERA_DE_SERVICIO. Las transiciones están en una tabla, así una transición inválida se rechaza con un solo punto de control.

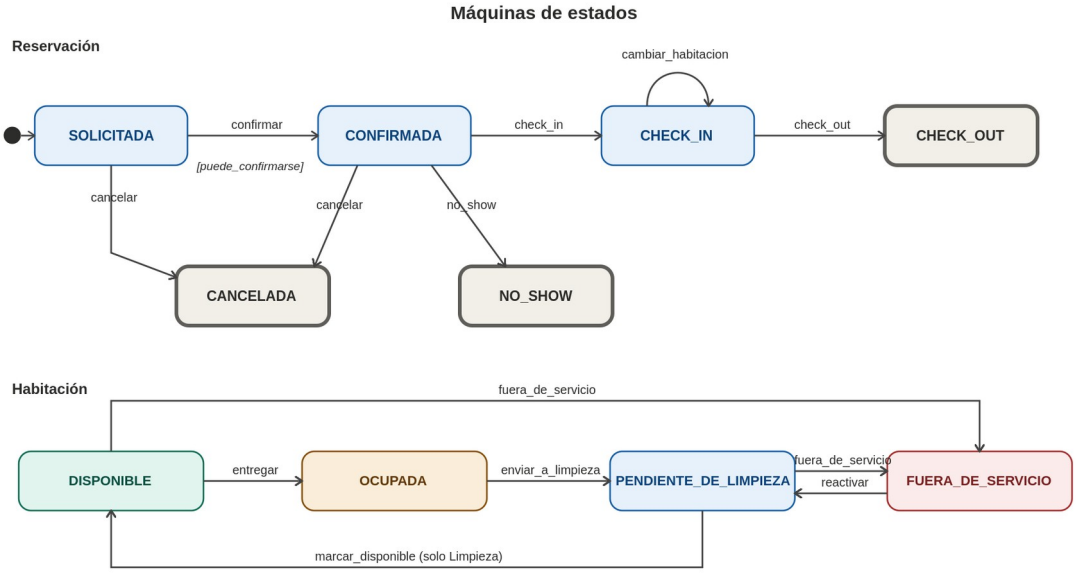


Figura 3. Máquinas de estados de la reservación y de la habitación.

6.4 Specification Pattern. Cada regla pequeña es una especificación que se combina con Y, O y No:

Especificación combinada	Composición
puede_procesarse	HuespedActivo Y DetalleNoVacio Y ReservacionAutorizada
puede_confirmarse	puede_procesarse Y CategoriasHabilitadas Y CupoSuficiente
beneficio_preferente	HuespedPreferente Y TotalMinimo(umbral) Y NO PromocionActiva
requiere_revision	NO HuespedActivo O NO ReservacionAutorizada

6.5 Diagrama de clases.

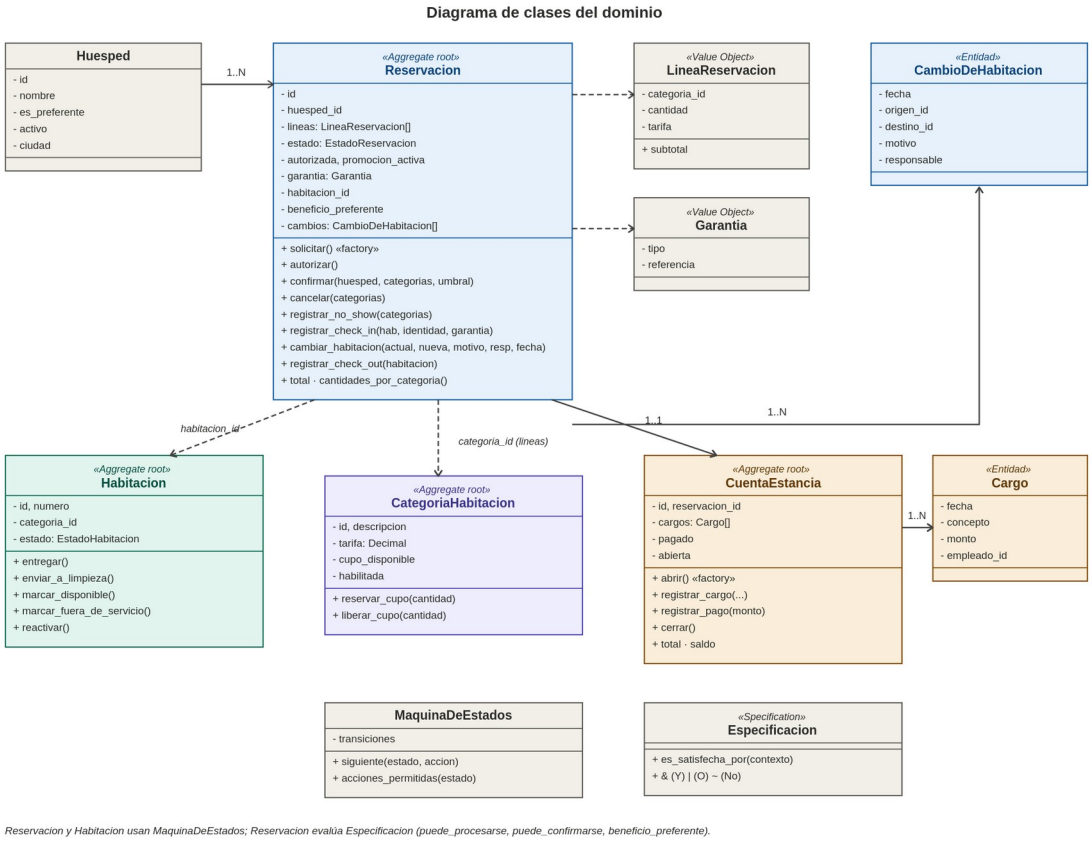


Figura 4. Diagrama de clases del dominio.

7. Arquitectura, patrones, SOLID y decisiones (ADR)

La dependencia apunta siempre hacia el dominio. El dominio no importa nada de las otras capas; la aplicación solo conoce contratos (puertos); la infraestructura los implementa; y composicion.py arma todo en un solo lugar.

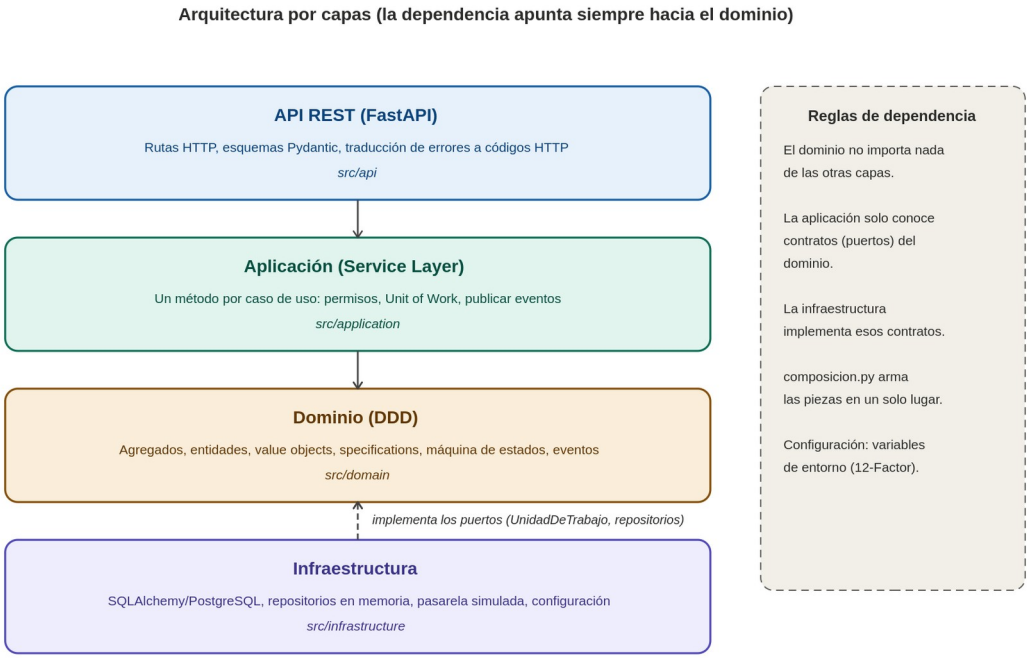


Figura 5. Arquitectura por capas.

Patrón o principio	Dónde y para qué
Repository	Contratos en el dominio; implementaciones SQL y en memoria. El dominio no conoce la base de datos.
Unit of Work	Cada caso de uso guarda todo o nada; un rechazo deja la información intacta.
Service Layer	Un servicio por actor y un método por caso de uso; solo coordinan.
CQRS básico	ServicioDeConsultas solo lee; los demás servicios modifican.
Specification y State Machine	Reglas combinables con Y, O y No; transiciones explícitas en una tabla.
Domain Events	Hechos como HabitacionDisponible o ReservacionCancelada (con penalidad) se publican después de guardar.
Ports and Adapters	La pasarela de garantías y el publicador de eventos se sustituyen por simulados en las pruebas.
SOLID	S: un servicio por actor. O: una regla nueva es una especificación nueva (el tope de crédito se agregó sin tocar otros servicios). L: memoria y SQL son intercambiables. I: puertos pequeños. D: los servicios dependen de contratos.

Decisiones de arquitectura (ADR)

ADR	Decisión	Motivo
001	Capas con diseño guiado por el dominio	Reglas repetidas en varios casos de uso.
002	API REST con FastAPI y OpenAPI	Validación y documentación integradas.
003	PostgreSQL, SQLAlchemy y repositorios por agregado	Dominio sin conocer la base de datos.
004	Unit of Work	Evitar estados a medias.
005	Specification Pattern	Reglas combinables y legibles.
006	Máquina de estados explícita	Transiciones fáciles de probar.
007	Eventos publicados después de guardar	No avisar de lo que no se guardó.

ADR	Decisión	Motivo
008	Acceso por rol con el encabezado X-Rol	Simple; limitación documentada (JWT pendiente).
009	Docker, Compose y 12-Factor	Entornos reproducibles y sin secretos en el código.
010	Datos del PIA solo para pruebas	No adoptar reglas ilustrativas como requisitos.

8. Implementación: API REST

La documentación interactiva (Swagger) se genera sola en /docs y el contrato real en /openapi.json; el contrato de diseño está en docs/api/openapi.yaml. Las rutas, salvo /health, requieren el encabezado X-Rol.

Método	Ruta	Roles	Errores principales
GET	/categorias	Todos	403
POST	/reservaciones	Reservaciones	403, 404, 422
POST	/reservaciones/{id}/autorizar	Reservaciones	403, 404, 409
POST	/reservaciones/{id}/confirmar	Reservaciones	403, 404, 409, 422
POST	/reservaciones/{id}/cancelar (CAMBIO: cuerpo opcional)	Reservaciones	403, 404, 409
POST	/reservaciones/{id}/no-show	Reserv., Recepción	403, 404, 409
POST	/reservaciones/{id}/check-in	Recepción	403, 409, 422, 503
POST	/reservaciones/{id}/cargos (CAMBIO: tope de crédito)	Recepción	403, 404, 409, 422
POST	/reservaciones/{id}/cambio-habitacion	Recepción	403, 404, 409
POST	/reservaciones/{id}/check-out	Recepción	403, 404, 409
POST	/habitaciones/{id}/limpieza-terminada	Limpieza	403, 404, 409
POST	/habitaciones/{id}/fuera-de-servicio	Administración	403, 404, 409
GET	/reportes/ocupacion	Administración	403

Código	Cuándo
200 / 201	Operación realizada / recurso creado (solicitud y cargos).
400	Valor inválido que detecta el dominio.
403	Rol faltante, desconocido o sin permiso.
404	No existe el elemento.
409	Conflicto con el estado: transición inválida, sin cupo, habitación no disponible, cuenta cerrada o con saldo, o (CAMBIO) tope de crédito excedido.
422	Cuerpo inválido o condiciones del negocio no cumplidas (identidad, garantía, autorización).
503	La pasarela de garantías no respondió.

Fragmentos del código nuevo (CAMBIO). Tope de crédito en la cuenta de la estancia:

```
TOPE_CREDITO = Decimal("3000")

@property
def credito_usado(self):
    consumos = sum((c.monto for c in self.cargos
                    if c.concepto != CONCEPTO_HOSPEDAJE), Decimal("0"))
    return max(consumos - self.pagado, Decimal("0"))

def registrar_cargo(self, concepto, monto, empleado_id, fecha,
```

```
                    autorizacion_administracion=False):
    ...
    excede = self.credito_usado + monto > TOPE_CREDITO
    if concepto != CONCEPTO_HOSPEDAJE and excede and not autorizacion_administracion:
        raise CreditoExcedidoError("El consumo supera el tope de crédito de 3000 MXN")
```

Penalidad de cancelación a 48 horas en la reservación:

```
def cancelar(self, categorias, ahora=None, fecha_llegada=None,
             autorizacion_jefe=False):
    estaba_confirmada = self.estado == EstadoReservacion.CONFIRMADA
    self.estado = MAQUINA_RESERVACION.siguiente(self.estado, "cancelar")
    penalidad = Decimal("0")
    if estaba_confirmada:
        self._liberar_cupo(categorias)
        limite = timedelta(hours=HORAS_CANCELACION_GRATUITA)
        tardia = (ahora is not None and fecha_llegada is not None
                  and fecha_llegada - ahora < limite)
        if tardia and not autorizacion_jefe and self.lineas:
            penalidad = self.lineas[0].tarifa  # una noche
    self.penalidad_cancelacion = penalidad
    self._registrar(ReservacionCancelada(self.id, penalidad))
    return penalidad
```

Secuencias de los flujos principales.

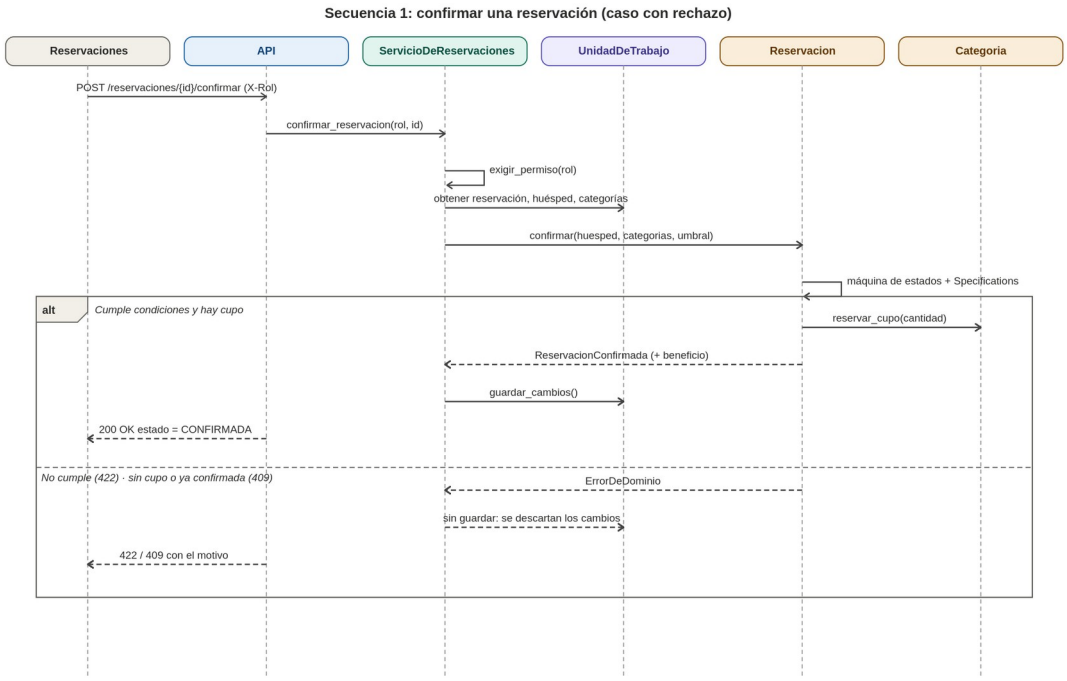


Figura 6. Secuencia: confirmar una reservación.

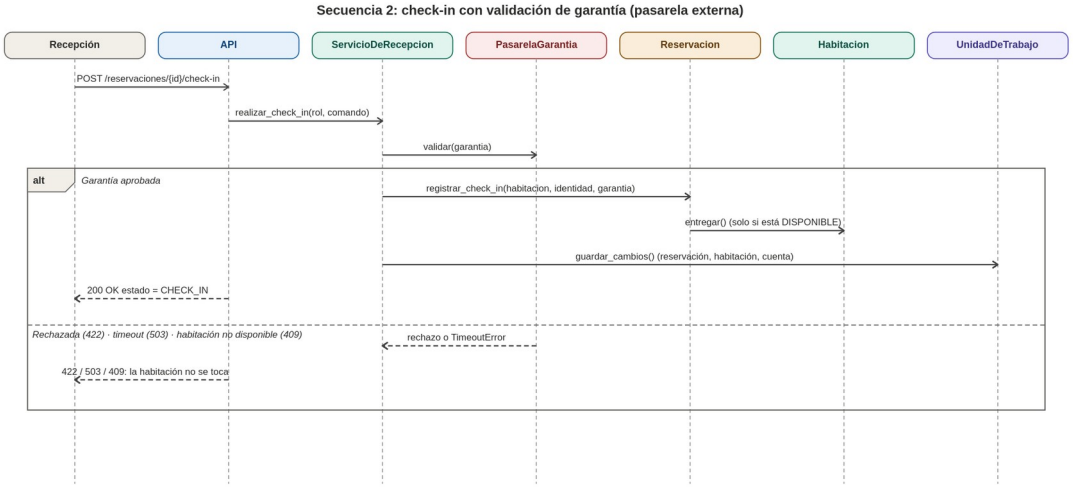


Figura 7. Secuencia: check-in con validación de garantía.

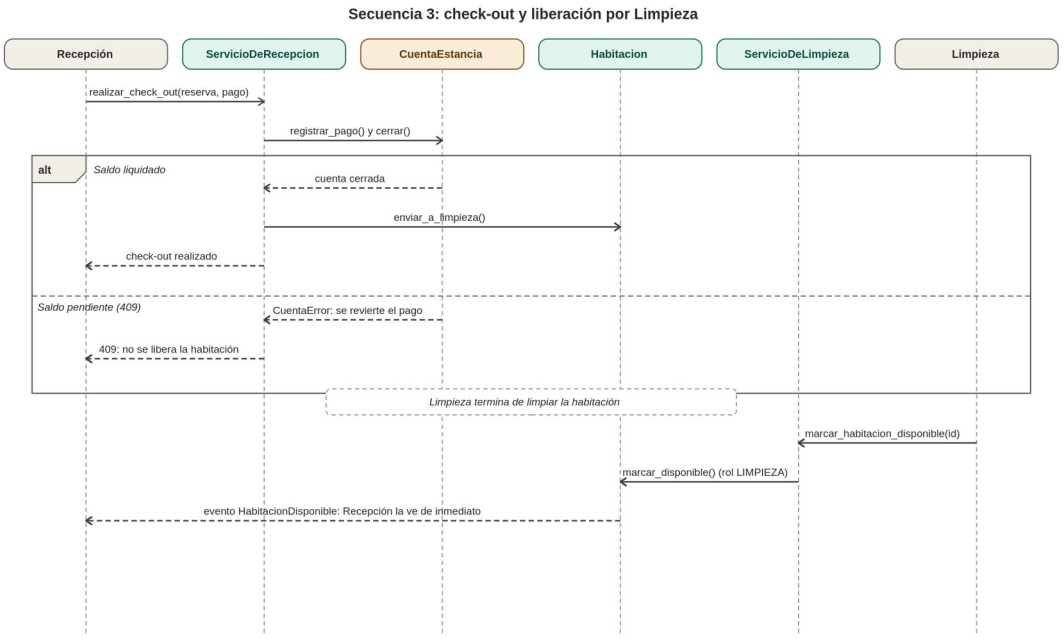


Figura 8. Secuencia: check-out y ciclo de limpieza.

9. Persistencia y configuración (12-Factor)

La base de datos tiene ocho tablas: huespedes, categorias, habitaciones, reservaciones, lineas_reservacion, cambios_habitacion, cuentas y cargos. Los importes usan Numeric(12,2) en la base y Decimal en el código. Las tablas se crean al arrancar; no hay migraciones. La penalidad de cancelación no se guarda (se devuelve en la respuesta y en el evento).

Variable de entorno	Uso
DATABASE_URL	Base de datos; memory usa memoria; postgres:// se normaliza al driver psycopg.
CARGAR_DATOS_DEMO	Carga 40 huéspedes, 20 categorías, 120 habitaciones y 60 reservaciones del paquete.
BENEFICIO_UMBRAL_MXN	Umbral del beneficio preferente (1500 por omisión).

Variable de entorno	Uso
RUTA_DATOS, HABITACIONES_POR_CATEGORIA	Carpeta de los CSV y habitaciones por categoría (6).
POSTGRES_DB / USER / PASSWORD	Credenciales del contenedor de base de datos; solo en .env (no se sube; existe .env.example sin secretos).

10. Pruebas y calidad

Tipo	Cantidad	Qué cubre
Dominio	89	Specifications, máquina de estados, agregados, cuenta, cupo y value objects.
Reglas de la Actividad 5 (CAMBIO)	13	Penalidad de cancelación a 48 h y tope de crédito de $3,000 (dominio y servicios).
Permisos por rol	15	Quién puede ejecutar cada operación y qué pasa con roles desconocidos.
Servicios (casos de uso)	32	Flujos completos en memoria, rechazos con reversión, pasarela con Mock y reporte.
Datos del PIA	10	120 casos y 20 transiciones contra un cálculo independiente.
API: errores, serializadores y configuración	22	Códigos HTTP, JSON y variables de entorno.
Trazabilidad	5	Cada historia, endpoint y prueba de la matriz existe.
Unitarias (verificadas)	186	Total que pasó en el entorno de elaboración.
Integración (por ejecutar)	23	SQLAlchemy sobre SQLite o PostgreSQL (8) y la API con TestClient (15).

Total esperado con pytest en el equipo: unas 209 pruebas. Resultado verificado de las unitarias (docs/evidencias/pytest_unit_verificado.txt):

```
.......................................................................... [ 77%]
........................................                                 [100%]
186 passed in 0.35s
```

10.1 Resultado de los 120 casos del PIA

(docs/evidencias/resultados_casos_prueba.csv)

Tipo de caso	Resultado	Cantidad
evaluar_beneficio	APLICA	10
evaluar_beneficio	NO APLICA	50
validar_operacion	CUMPLE	42
validar_operacion	NO CUMPLE	18

10.2 Resultado de las 20 transiciones

Acción	Resultado	Cantidad
aprobar	OK	1
aprobar	RECHAZADA	3
cancelar	OK	4
completar	OK	2
completar	RECHAZADA	2
repetir_aprobacion	RECHAZADA	4
saltar_estado	RECHAZADA	4

Cada resultado se compara con un cálculo independiente hecho directamente sobre los CSV (tests/unit/test_datos_pia.py), así la prueba no repite el camino del código que evalúa. Las transiciones se interpretaron así: aprobar = confirmar; completar = confirmar, check-in y check-out; repetir_aprobacion = confirmar dos veces (la segunda debe fallar); saltar_estado = check-in sin confirmar (debe fallar).

10.3 Trazabilidad. docs/matriz_trazabilidad.csv relaciona necesidad, historia, requisito, endpoint, caso de uso y pruebas. La prueba test_trazabilidad.py comprueba que todo lo citado existe, así la matriz no queda desactualizada sin que las pruebas fallen.

10.4 Cómo reproducir.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pytest -v --cov=src --cov-report=term-missing --cov-
fail-under=80
.\.venv\Scripts\python.exe -m ruff check src tests scripts
.\.venv\Scripts\python.exe scripts\ejecutar_casos_pia.py
```

11. Docker, CI/CD, despliegue y evidencias

Servicio de Compose	Función
db	PostgreSQL 16 con healthcheck y una segunda base (hotel_pruebas) para pruebas.
api	Imagen de producción: usuario sin privilegios, sin herramientas de prueba; espera a que la base esté sana.
pruebas	Perfil opcional: ejecuta pytest con cobertura contra PostgreSQL.

Trabajo de GitHub Actions	Qué hace
calidad-y-pruebas	Ruff, pytest con cobertura mínima de 80 % y reporte HTML como artefacto.
integracion-postgres	Pruebas de persistencia contra PostgreSQL real.
imagen-docker	Construye la imagen de producción.
despliegue	En main, dispara el despliegue en Render si existe el secreto RENDER_DEPLOY_HOOK_URL.

Pasos para desplegar: subir el repositorio a GitHub, crear un Blueprint en Render apuntando al repositorio (lee render.yaml), verificar /health y /docs, y guardar el Deploy Hook como secreto de GitHub.

```
docker compose up --build
docker compose --profile pruebas run --rm pruebas
```

URL pública (Render): [pegar aquí la URL cuando se despliegue] · **Repositorio:** https://github.com/jorgekingreate5/hotel-costa-azul-cln

Evidencias que debe agregar el equipo (no se pueden generar desde el repositorio):

CAPTURA PENDIENTE – P1: pytest con cobertura (python -m pytest --cov=src --cov-fail-under=80) mostrando todas las pruebas en verde y el porcentaje.

CAPTURA PENDIENTE – P2: ruff check sin errores.

CAPTURA PENDIENTE – P3: GitHub Actions con todos los trabajos en verde.

CAPTURA PENDIENTE – P4: docker compose up --build con los servicios db y api levantados.

CAPTURA PENDIENTE – P5: docker compose --profile pruebas run --rm pruebas con las pruebas en verde.

CAPTURA PENDIENTE – P6: Swagger (/docs) con los endpoints y una respuesta 409 de CreditoExcedidoError al registrar un cargo sobre el tope.

CAPTURA PENDIENTE – P7: Render: la URL pública con /health y /docs funcionando.

CAPTURA PENDIENTE – P8: Historial de commits en GitHub con los tres integrantes.

12. Uso de inteligencia artificial

Se usó Claude (Anthropic) como apoyo para analizar el paquete del PIA, proponer el diseño, escribir código, pruebas y documentación. Las decisiones y supuestos quedaron documentados para que el equipo los revise y los defienda.

Situación	Revisión	Decisión
El paquete trae reglas ilustrativas (umbral, flujo de estados).	No son requisitos del negocio.	Se usan para pruebas y quedan como supuestos configurables (ADR-010).
Faltaban las reglas del BRD del equipo (cancelación a 48 h y tope de crédito).	Eran requisitos reales del caso y de la Actividad 5.	Se integraron al PIA y se marcaron como CAMBIO.
Probar con un oráculo igual al código sería circular.	El resultado coincidiría siempre.	El oráculo se calcula directo de los CSV, con otro camino de código.
No había FastAPI, SQLAlchemy, Ruff ni Docker en el entorno de elaboración.	No se podía ejecutar esa parte.	Se declaró como pendiente y se verificó lo demás con pytest y trace.

CAPTURA PENDIENTE – Agregar aquí los prompts propios del equipo, las correcciones que hicieron y qué entendieron de cada parte del código.

13. Limitaciones y trabajo futuro

- Autenticación real (JWT) en lugar del encabezado X-Rol (ADR-008).

- Eventos persistentes con una tabla outbox (ADR-007) y migraciones con Alembic.
- Guardar la fecha de llegada y la penalidad de cancelación en la base de datos (CAMBIO: hoy viajan en la petición y en la respuesta).
- Expiración de la pre-reserva a 24 h, no-show con cargo, salida anticipada, extensión y llegada anticipada (US-20 a US-22).
- Pasarela de garantías real y reportes adicionales para Administración.

Conclusiones:

El PIA integró en un solo proyecto lo trabajado en las unidades anteriores: el BRD y las historias de usuario definieron qué construir, el diseño por capas con DDD separó las reglas del negocio de la tecnología, y la API, la base de datos, Docker y GitHub Actions convirtieron ese diseño en algo que se puede ejecutar, probar y desplegar. Las reglas de reservación, estancia y limpieza viven en el dominio, se prueban sin base de datos y se exponen mediante una API con códigos de error coherentes (400, 403, 404, 409, 422 y 503). Los cuatro problemas del caso tienen una respuesta concreta y probada.

Integrar las reglas de la Actividad 5 mostró el valor de la arquitectura: el tope de crédito de $3,000 MXN y la penalidad de cancelación a 48 horas se agregaron como cambios pequeños en el agregado correspondiente, con sus pruebas de éxito, error y límite (por ejemplo, exactamente 48 horas es gratis y 47 h 59 min cobra una noche; exactamente $3,000 se permite y un centavo más se rechaza), sin tocar el resto del sistema. También quedó claro que cada regla debe poder rastrearse: la matriz de trazabilidad une la necesidad del BRD con la historia, el endpoint, el caso de uso y las pruebas, y una prueba automática evita que se desactualice.

Aprendizajes principales: una regla de negocio se protege mejor cuando es pequeña, explícita y tiene una prueba; las pruebas con un oráculo independiente detectan errores que una prueba circular no vería; y declarar con honestidad qué se verificó y qué falta ejecutar es parte de la calidad del trabajo. Lo que resta no es diseño sino ejecución en el entorno del equipo (pruebas de integración, Docker, GitHub Actions, despliegue y capturas) y la validación con el negocio de las políticas que siguen abiertas.

Conclusiones individuales:

Jorge Luis Molina García: Entendí que el valor de separar el dominio de la API y la base de datos se nota al agregar una regla nueva: el tope de crédito cambió un solo agregado y sus pruebas. También aprendí a escribir las pruebas de límite (exactamente 48 horas y exactamente $3,000) para no depender de la intuición.

Jonathan Alexander Reyes Garcia: Lo más útil fue ver cómo Specification y la máquina de estados vuelven legibles las reglas: puede_procesarse y beneficio_preferente se leen casi como el BRD. Entendí por qué el oráculo de las

pruebas del PIA se calcula directo de los CSV y no con el mismo código que se está probando.

Leonardo Davila Gongora: Aprendí la importancia de la trazabilidad y de la automatización: la matriz que une necesidad, historia, endpoint y prueba, y el pipeline de GitHub Actions que rechaza cambios que rompen las pruebas. También entendí por qué Docker Compose con variables de entorno permite que cualquiera levante el sistema igual.

Link de GitHub:

https://github.com/jorgekingreate5/hotel-costa-azul-cln (repositorio del PIA: código, pruebas, documentación, Docker y CI/CD). URL pública de la API en Render: [pegar aquí al desplegar].